Informe del Desafío II

Amigos para siempre, con UdeABook

Juan José Mosquera López
juan.mosqueral@udea.edu.co

Juan Esteban Franco Aranzales
juan.francoa@udea.edu.co

Universidad de Antioquia, Facultad de Ingeniería
Informática II – Semestre 2026-2
Octubre de 2026

Índice

1. Introducción **4**

2. Análisis del problema **4**

2.1. Contexto y requisitos funcionales 4

2.2. Entidades y relaciones 4

2.3. Restricciones del desarrollo 5

2.4. Supuestos y aclaraciones 5

3. Diseño de la solución **5**

3.1. Arquitectura y diagrama de clases 5

3.2. Decisiones de diseño 6

3.3. Estructuras de datos y eficiencia 7

3.4. Formato de los archivos de datos 7

4. Algoritmos esenciales **8**

4.1. Carga de datos 8

4.2. Inicio de sesión 9

4.3. Mejores amigos con probabilidad ponderada 9

4.4. Sugerencias de amistad 9

4.5. Administración de amistades 10

4.6. Visita al perfil de un amigo 10

4.7. Mensajería 10

4.8. Guardado de datos 10

4.9. Medición de recursos 10

5. Gestión de la memoria dinámica **11**

6. Pruebas y resultados **11**

6.1. Datos de prueba 11

6.2. Verificación 11

6.3. Consumo de recursos observado 12

6.4. Ejecución 12

7. Problemas de desarrollo **12**

8. Evolución de la solución **13**

9. Limitaciones y trabajo futuro **14**

10. Uso de herramientas de inteligencia artificial **14**

11. Conclusiones **14**

1. Introducción

UdeABook es una red social cuyo núcleo lógico se desarrolló en C++ bajo el paradigma de programación orientada a objetos. El programa gestiona usuarios, amistades, solicitudes de amistad, mensajes, álbumes y fotos, y ofrece sus funcionalidades mediante un menú en consola. Los datos se leen del almacenamiento permanente al iniciar y se actualizan al cerrar sesión.

El desarrollo estuvo guiado por tres criterios que el enunciado enfatiza: el uso de estructuras de datos de creación propia y memoria dinámica (sin contenedores de la STL, sin `struct` y sin cadenas de la biblioteca estándar), la eficiencia medida en iteraciones y memoria, y un diseño modular con una clase por responsabilidad. Este informe documenta el análisis del problema, el diseño, los algoritmos esenciales, las pruebas realizadas, los problemas encontrados y la evolución de la solución.

2. Análisis del problema

2.1. Contexto y requisitos funcionales

El cliente solicita las siguientes funcionalidades, que se mantuvieron con la numeración del enunciado.

I. **Carga y actualización de datos.** Lectura y escritura desde y hacia archivos. No aparece en el menú.

II. **Ingreso y mejores amigos.** Inicio de sesión y presentación de tres fotos de perfil de amigos, elegidas al azar con probabilidad ponderada por la cantidad de amigos en común (peso 1, 2.5 y 3.5), con opción de refrescarlas.

III. **Administrar amistades.** Enviar solicitudes y aceptar las recibidas.

IV. **Visitar el perfil de un amigo.** Listar sus álbumes públicos o de amigos y recorrer las fotos de uno de ellos en forma de carrusel, con la opción de dar o quitar like.

V. **Sugerencias de amistad.** Mostrar K sugerencias ($1 \leq K \leq 20$) ordenadas por amigos en común, con la opción de enviar solicitud.

VI. **Mensajes.** Enviar, leer los nuevos y buscar por rango de fechas.

VII. **Medición del consumo de recursos.** Mostrar, al terminar cada funcionalidad, las iteraciones realizadas y la memoria en uso.

2.2. Entidades y relaciones

Del enunciado se identificaron las entidades *Usuario*, *Álbum*, *Foto*, *Mensaje* y *Solicitud de amistad*, además de una entidad que coordina las demás (*Plataforma*). Las relaciones de pertenencia son jerárquicas: la plataforma contiene usuarios, cada usuario contiene álbumes, y cada álbum contiene fotos. La amistad es una relación simétrica de muchos a muchos entre usuarios: si A es amigo de B, entonces B es amigo de A. Un mensaje relaciona a un emisor con un receptor, y una solicitud pendiente relaciona a quien la envía con quien la recibe.

2.3. Restricciones del desarrollo

- Lenguaje C++ con POO, programación modular en archivos de cabecera y de implementación, y al menos tres clases.
- Sin contenedores de la STL, sin `struct` y sin librerías de terceros que resuelvan asuntos esenciales. Solo se emplea la librería de números aleatorios. Las estructuras de datos son propias y usan memoria dinámica.
- El almacenamiento permanente no puede usarse como área de procesamiento o intercambio: los archivos se leen al inicio y se escriben al cierre.
- Aplicación de sobrecarga de operadores, funciones amigas, constructores de copia y plantillas.
- Demostración con al menos 1000 usuarios registrados.

2.4. Supuestos y aclaraciones

El enunciado deja varios puntos sin precisar. Se adoptaron los siguientes supuestos, que se dejan documentados para poder discutirlos con el cliente.

- **Credenciales.** El inicio de sesión exige contraseña, pero esta no figura entre los datos del usuario. Se asume que cada usuario tiene una contraseña almacenada junto a sus datos.
- **Rangos de prioridad.** Los rangos dados son 10 o menos, de 11 a 20 y "más de 21" amigos en común, de modo que el valor 21 queda sin cubrir. Se asume que el rango superior comienza en 21. Los amigos en común se cuentan entre el usuario que inició sesión y cada uno de sus amigos.
- **Likes.** Para poder quitar un like es necesario saber quiénes lo dieron; por eso cada foto conserva, además del conteo, la lista de usuarios que la marcaron.
- **Orden de los mensajes.** Se asume que el orden por fecha de recepción coincide con el orden por fecha de envío.
- **Longitud del mensaje.** El límite de 512 caracteres se verifica contando bytes del texto ingresado.
- **Submenú de amistades.** El enunciado menciona tres opciones pero describe dos (enviar y aceptar solicitudes); se implementaron esas dos más la opción de volver.
- **Fecha actual.** Las solicitudes y los mensajes nuevos se fechan con la fecha del sistema en hora de Colombia (UTC-5). Si esa fecha fuera anterior a la del último mensaje del destinatario, se usa esta última para conservar el orden cronológico.

3. Diseño de la solución

3.1. Arquitectura y diagrama de clases

La solución se organiza en doce clases, más un componente externo. La Figura 1 muestra la versión resumen del diagrama y la Figura 2 la versión expandida con atributos y métodos, ambas en la

notación UML simplificada del curso.

[Insertar aquí la figura: diagrama_resumen.pdf]

Figura 1: Diagrama de clases, versión resumen.

[Insertar aquí la figura: diagrama_expandido.pdf]

Figura 2: Diagrama de clases, versión expandida.

Plataforma
Coordina el sistema: carga y guarda los datos, autentica, ejecuta el menú y las funcionalidades. Posee la lista de usuarios y la matriz de amistad.

Usuario
Datos personales, álbumes, mensajes enviados y recibidos, y solicitudes pendientes. Es dueño de sus álbumes, de los mensajes que envía y de las solicitudes que recibe.

Album y Foto
Un álbum agrupa fotos y define su privacidad (público, privado o solo amigos). Cada foto guarda su fecha, tamaño, ruta y los usuarios que le dieron like.

Mensaje y Solicitud
Representan, respectivamente, un texto enviado entre amigos y una petición de amistad pendiente.

MatrizAmistad
Matriz de adyacencia de bits que representa la amistad entre todos los usuarios.

Medidor
Contadores de iteraciones y memoria para la funcionalidad VII.

LectorLinea
Separa los campos de cada línea de los archivos de datos.

Texto, Fecha y Lista<T>
Tipos de apoyo propios: cadena de caracteres, fecha del calendario y arreglo dinámico genérico.

3.2. Decisiones de diseño

- **Propiedad de los objetos.** Cada objeto tiene un único dueño que lo libera. Los usuarios son dueños de sus álbumes, mensajes enviados y solicitudes; los mensajes recibidos se guardan como punteros a los mensajes del emisor, lo que evita duplicar su contenido.
- **Tipos de apoyo sin líneas en el diagrama.** `Texto`, `Fecha` y `Lista<T>` aparecen como tipo de atributo en casi todas las clases. Dibujar todas esas relaciones haría ilegible el diagrama, por lo que se omiten.
- **Medidor con miembros estáticos.** Los contadores son de clase, de modo que cualquier ciclo del programa, incluidos los de la plantilla de listas, puede reportar iteraciones sin recibir referencias adicionales.
- **Amistad fuera del usuario.** La relación de amistad no se almacena en `Usuario` sino en `MatrizAmistad`, indexada por la posición de cada usuario.

- **Sin getters ni setters en el diagrama.** El estándar del curso permite omitirlos en diagramas grandes; en el código sí existen.
- **Cadena propia.** Como la cadena de la biblioteca estándar no está permitida, se implementó la clase `Texto`, con constructores, destructor, copia, comparación, concatenación y lectura de caracteres.
- **Concentración en Plataforma.** La clase `Plataforma` quedó con muchas responsabilidades (carga, guardado, menú y funcionalidades). Una mejora natural sería separar una clase de menú y otra de persistencia.

3.3. Estructuras de datos y eficiencia

Para cada operación dominante se eligió la estructura que la hace más barata. La Tabla 1 resume la elección, donde n es el número de usuarios, d la cantidad de amigos de un usuario, m la cantidad de mensajes de un usuario y k el tamaño del resultado.

Necesidad	Estructura	Costo
Buscar un usuario por nombre	Arreglo dinámico ordenado y búsqueda binaria	$O(\log n)$
Saber si dos usuarios son amigos	Matriz de bits	$O(1)$
Amigos en común de dos usuarios	AND de filas y conteo de bits	$O(n/64)$
Mensajes de un rango de fechas	Lista cronológica y dos búsquedas binarias	$O(\log m + k)$
Leer mensajes nuevos	Índice del primer no leído	$O(k)$
Recorrer fotos de un álbum	Índice con aritmética modular	$O(1)$ por paso
Eliminar un like o una solicitud	Intercambio con el último elemento	$O(1)$ tras ubicarlo

Cuadro 1: Estructuras elegidas y costo de la operación dominante.

Amistad como matriz de bits. Con 1000 usuarios, una matriz de valores booleanos ocuparía aproximadamente 1 MB. Empaquetando cada fila en palabras de 64 bits, la matriz completa ocupa unos 125 KB y permite contar los amigos en común procesando 64 usuarios por operación. La principal desventaja es que el espacio crece con el cuadrado del número de usuarios, lo cual es aceptable para el tamaño de la demostración pero constituye una limitación para escalas mayores. El conteo de bits se escribió a mano, sin ciclos, para no depender de funciones específicas del compilador.

Mensajes en orden cronológico. Los mensajes de cada usuario se almacenan en el orden en que llegan, que coincide con el orden por fecha. Esto evita ordenar y permite buscar rangos con búsqueda binaria. Como la lectura de mensajes nuevos muestra todos los no leídos y luego los marca, los no leídos siempre ocupan el tramo final de la lista, y basta conservar la posición del primero.

3.4. Formato de los archivos de datos

Los datos se almacenan en seis archivos de texto plano, un registro por línea y campos separados por punto y coma. Las fechas se escriben como `dd/mm/aaaa`. Los usuarios se identifican por su

posición (comenzando en cero) en el archivo de usuarios, lo que hace compactos y rápidos de cargar los demás archivos.

usuarios.txt
`nombreUsuario;contraseña;nacimiento;correo;rutaFoto;ciudad;país;inscripción`. Debe estar ordenado por nombre y sin repetidos, porque la búsqueda binaria lo exige.

amistades.txt
`i;j`, con $i < j$, una línea por pareja de amigos.

solicitudes.txt
`receptor;emisor;fecha`.

mensajes.txt
`emisor;receptor;fecha;leído;texto`. El texto va al final, de modo que puede contener punto y coma. Para cada receptor, los mensajes deben estar en orden cronológico y los no leídos después de los leídos.

albumes.txt
`dueño;nombre;privacidad;ruta`, con privacidad `P` (público), `A` (amigos) o `R` (privado).

fotos.txt
`dueño;númeroDeÁlbum;fecha;tamaño;ruta;likes`, donde los likes son las posiciones de los usuarios, separadas por comas.

Al cargar, el programa valida que los índices existan, que los usuarios estén ordenados, que no haya líneas excesivamente largas que se trunquen en silencio y que los mensajes cumplan las condiciones de orden. Ante cualquier inconsistencia informa el archivo afectado y no continúa con datos incompletos. Al cerrar sesión se reescriben únicamente los archivos que el enunciado permite modificar: amistades, solicitudes, mensajes (incluido su estado leído) y likes. Los mensajes se guardan agrupados por receptor, siguiendo su lista de recibidos, lo que preserva su orden y la posición de los no leídos. Los archivos de usuarios y de álbumes no se alteran.

4. Algoritmos esenciales

Se describe la lógica en alto nivel; el código fuente está intra-documentado.

4.1. Carga de datos

1. Leer `usuarios.txt` línea por línea, creando un usuario por cada una y verificando que los nombres estén en orden estricto.
2. Crear la matriz de amistad con tantas filas como usuarios.
3. Leer las amistades y marcar ambos sentidos en la matriz.
4. Leer álbumes y fotos, asignando cada álbum a su dueño y cada foto a su álbum; por cada like de la foto, registrarlo.
5. Leer las solicitudes y guardarlas en la lista del receptor.

6. Leer los mensajes: cada uno se guarda en la lista de enviados de su emisor y se referencia desde la de recibidos de su receptor, verificando el orden cronológico y la posición de los no leídos.
7. Para cada usuario, ubicar el primer mensaje no leído.

4.2. Inicio de sesión

Se busca el usuario por nombre con búsqueda binaria y se compara la contraseña. Si coinciden, se activa la sesión y se reinicia el registro del grupo de fotos mostrado.

4.3. Mejores amigos con probabilidad ponderada

1. Obtener los amigos del usuario desde la matriz.
2. Para cada amigo, contar los amigos en común con el usuario y asignar un peso: 10 o menos, peso 2; de 11 a 20, peso 5; 21 o más, peso 7. Estos valores equivalen a 1, 2.5 y 3.5 pero permiten trabajar con enteros.
3. Excluir a los amigos del grupo mostrado anteriormente, siempre que queden suficientes amigos para formar un grupo nuevo.
4. Repetir tres veces el método de la ruleta sin reemplazo: sumar los pesos de los candidatos disponibles, generar un número aleatorio menor que la suma, recorrer los candidatos acumulando pesos hasta superarlo, elegirlo y retirarlo del sorteo.
5. Mostrar las rutas de las fotos de perfil elegidas.

Con menos de tres amigos se muestran todos; sin amigos se informa que no hay fotos para mostrar. Inicialmente se consideró recorrer todos los amigos antes de repetir alguno, pero esa estrategia iguala las probabilidades a largo plazo y anula la prioridad, por lo que se descartó. En una prueba con 60 000 grupos generados para un usuario con 29 amigos, quienes tienen entre 11 y 20 amigos en común aparecieron cerca de 2.3 veces más que quienes tienen 10 o menos, un valor algo inferior a 2.5 porque excluir el grupo anterior suaviza la diferencia.

4.4. Sugerencias de amistad

1. Para cada usuario que no sea el propio ni uno de sus amigos, calcular los amigos en común con el usuario mediante la matriz.
2. Descartar a quienes tienen cero amigos en común, pues no son amigos de sus amigos.
3. Seleccionar los K mayores con K pasadas sobre los candidatos, sin ordenar la totalidad; los empates se resuelven por el menor índice.
4. Mostrar la lista y ofrecer enviar una solicitud a una de ellas.

El costo es del orden de $n \cdot n/64 + K \cdot n$ operaciones.

4.5. Administración de amistades

Al enviar una solicitud se valida que el destinatario exista, que no sea el propio usuario, que no sean ya amigos, que no exista una solicitud previa del usuario hacia él y que él no haya enviado ya una solicitud al usuario (en ese caso se indica que debe aceptarla). Al aceptar una solicitud, se marca la amistad en la matriz en ambos sentidos y se retira la solicitud de la lista de pendientes.

4.6. Visita al perfil de un amigo

Se verifica que el nombre corresponda a un amigo. Se listan solo los álbumes con privacidad pública o de amigos. Al elegir uno se entra en el carrusel: se muestra la ruta de la foto actual, su conteo de likes y si el usuario ya le dio like, y se ofrecen las opciones siguiente, previa, dar o quitar like y salir. El avance y el retroceso usan aritmética modular, de modo que después de la última foto se vuelve a la primera y antes de la primera se pasa a la última.

4.7. Mensajería

Enviar
Verificar que el destinatario exista y sea amigo, que el texto no esté vacío ni contenga solo espacios y que no supere 512 caracteres. El mensaje se agrega a la lista de enviados del emisor y se referencia desde la de recibidos del receptor.

Leer nuevos
Recorrer desde el primer mensaje no leído hasta el final, mostrando el emisor y el texto de cada uno, y marcarlos como leídos, lo que también se refleja en la lista de enviados del emisor.

Rango de fechas
Validar ambas fechas y que el fin no sea anterior al inicio. Con una búsqueda binaria se ubica el primer mensaje con fecha mayor o igual al inicio, con otra el primero con fecha mayor al fin, y se listan los comprendidos entre ambos, con extremos incluidos.

4.8. Guardado de datos

Al cerrar sesión se recorren los usuarios y se reescriben los cuatro archivos modificables en el mismo formato en que se leen. Las amistades se escriben recorriendo las filas de la matriz y conservando solo las parejas con $i < j$.

4.9. Medición de recursos

Iteraciones. Cada ciclo del programa, incluidos los de las clases de apoyo (copia de listas, comparación y concatenación de textos, lectura de campos), incrementa el contador compartido. Las funcionalidades que usan el generador aleatorio reportan además el número de invocaciones.

Memoria. Cada clase calcula el espacio que ocupa: el tamaño del propio objeto, más sus bloques de memoria dinámica, más lo que ocupan los objetos que posee. La plataforma suma estos valores para todas las estructuras y a ello agrega el tamaño de las variables locales y parámetros por

valor de la funcionalidad en ejecución. Mientras se calcula la memoria el conteo de iteraciones se suspende, para que medir no altere las iteraciones de la funcionalidad. Los mensajes recibidos no se suman, porque ya los cuenta su emisor. Se trata de una medición cuidadosa pero aproximada: no incluye el costo interno del administrador de memoria ni los datos del sistema.

5. Gestión de la memoria dinámica

- **Liberación.** Cada clase que reserva memoria con `new` la libera en su destructor: la plataforma libera usuarios y matriz; el usuario, sus álbumes, mensajes enviados y solicitudes; el álbum, sus fotos; y las clases de apoyo, sus bloques de datos.
- **Copia.** Las clases con memoria dinámica implementan constructor de copia y operador de asignación con copia profunda, y se protegen contra la autoasignación. La plataforma no es copiable.
- **Verificación.** Las pruebas se ejecutaron con detectores de errores de memoria (accesos fuera de límites, uso tras liberar, fugas y comportamiento indefinido), sin que se reportara ningún hallazgo.
- **Validación de la reserva.** En C++ una reserva con `new` que falla no devuelve un puntero nulo, sino que lanza la excepción `std::bad_alloc`. El programa principal la captura, informa que no hay memoria suficiente y termina de forma controlada con un código de salida distinto de cero, en lugar de cerrarse abruptamente. Se probó limitando artificialmente la memoria disponible. Como consecuencia, los cambios no guardados de la sesión en curso se pierden; por ello el guardado se realiza en cada cierre de sesión.

6. Pruebas y resultados

6.1. Datos de prueba

Se desarrolló un programa aparte, que no forma parte de la solución, para generar datos reproducibles con semilla fija. El conjunto principal contiene 1200 usuarios, 22 000 amistades (unos 37 amigos por usuario), 400 solicitudes, 19 770 mensajes entre el 1 de agosto y el 12 de octubre de 2026, 2993 álbumes y 19 502 fotos. Las amistades se generaron en comunidades densas con enlaces aleatorios entre ellas, de modo que existan amistades con pocos, medianos y muchos amigos en común (aproximadamente 2900, 9900 y 9100 amistades en cada rango), lo que permite ejercitar los tres pesos de la prioridad.

6.2. Verificación

- **Comparación independiente.** Las 12 sugerencias de un usuario, los conteos de mensajes por rango de fechas y los mensajes sin leer se compararon con cálculos hechos aparte, directamente sobre los archivos, y coincidieron exactamente.
- **Persistencia.** Se hicieron cambios en un proceso (solicitud, mensaje, lectura de mensajes y like), se cerró el programa y, en procesos nuevos, se verificó que la solicitud pudiera aceptarse, que el mensaje llegara a su destinatario, que los mensajes leídos no reaparecieran y que

el like y la amistad persistieran. Al guardar sin cambios, cuatro de los seis archivos quedaron idénticos y los otros dos con el mismo contenido en distinto orden.

- **Casos límite.** Usuarios con 0, 1, 2, 3, 4 y 6 amigos; álbumes privados, vacíos y mixtos; valores de K fuera de rango o no numéricos; mensajes de 512 y 513 caracteres, vacíos o con tildes, eñes, comas y puntos y comas; entradas de consola vacías o excesivamente largas.
- **Archivos dañados.** Archivos ausentes o vacíos, usuarios repetidos o desordenados, índices fuera de rango, negativos o con letras, referencias a álbumes o usuarios inexistentes, mensajes desordenados, no leídos antes de leídos, líneas excesivamente largas, líneas en blanco y fin de línea de Windows.

6.3. Consumo de recursos observado

La Tabla 2 resume valores observados con los 1200 usuarios; cambian ligeramente entre ejecuciones y según el usuario.

Funcionalidad	Iteraciones	Memoria (bytes)
Carga de datos	11 344 793	5 063 949
Inicio de sesión	135 a 152	aprox. 5 064 000
Mejores amigos (3 invocaciones del generador)	1000 a 1300	aprox. 5 064 300
Enviar solicitud de amistad	120 a 150	aprox. 5 063 950
Buscar mensajes por rango de fechas	35 a 45	aprox. 5 064 250
Leer mensajes nuevos	2 a 8	aprox. 5 064 000
Visita al perfil de un amigo (dos álbumes)	aprox. 230	aprox. 5 064 200
Guardado al cerrar sesión	aprox. 661 000	aprox. 5 064 000

Cuadro 2: Consumo de recursos observado.

La carga y el guardado son las operaciones más costosas porque recorren todos los registros de los archivos, y el costo de la carga incluye el trabajo carácter por carácter de las clases de texto. Las funcionalidades interactivas cuestan entre unas pocas y algunos miles de iteraciones gracias a las estructuras elegidas, y la memoria total se mantiene cercana a 5 MB, donde la mayor parte corresponde a los mensajes, las fotos y las cadenas.

6.4. Ejecución

El proyecto se desarrolló con Qt Creator, usando únicamente el entorno de desarrollo y no las clases de Qt. Los seis archivos de datos deben encontrarse en el directorio de trabajo de la ejecución o bien indicarse su ruta en el programa principal. Para las pruebas se usan cuentas de la forma `user0001` con contraseña `pass0001`, hasta `user1200`.

7. Problemas de desarrollo

- **Prohibición de la cadena estándar.** Se descubrió durante el desarrollo que la cadena de la biblioteca estándar no estaba permitida. Gracias a que el tipo de texto estaba aislado en un solo archivo, el cambio por la clase propia `Texto` no requirió modificar las demás clases.

- **Plantillas y archivos de implementación.** El asistente del entorno generó un archivo de implementación para `Lista<T>`, lo que produjo errores de compilación, porque el compilador necesita ver el código completo de una plantilla. Se resolvió eliminando ese archivo y dejando la implementación en la cabecera.
- **Dos programas en un mismo proyecto.** El generador de datos, que tiene su propio punto de entrada, se incluyó por error en el proyecto de la solución, lo que produjo una definición duplicada de la función principal. Se separó en un proyecto aparte.
- **Directorio de trabajo.** El programa no encontraba los archivos de datos porque el entorno ejecuta desde la carpeta de compilación y no desde la del código. Se aclaró cómo configurar el directorio de trabajo o la ruta.
- **La medición alteraba lo medido.** Calcular la memoria recorre las estructuras y esos ciclos se sumaban a las iteraciones de la funcionalidad. Se agregó la posibilidad de pausar el conteo durante la medición.
- **Prioridad aplanada.** La primera idea para evitar repeticiones en las fotos de mejores amigos, recorrer todos antes de repetir, igualaba las probabilidades y anulaba la ponderación. Se reemplazó por la exclusión únicamente del grupo anterior.
- **Defectos hallados al probar casos límite.** Una línea larga escrita en consola cerraba el programa, los números con signo o letras se aceptaban, un archivo de mensajes desordenado se cargaba sin aviso, un mensaje no leído antes de uno leído nunca se mostraba y una línea demasiado larga cortaba la carga en silencio. Todos se corrigieron y se repitió la batería de pruebas completa.
- **Falta de validación de la reserva de memoria.** Inicialmente ninguna reserva se verificaba; se agregó la captura de la excepción correspondiente en el programa principal.

8. Evolución de la solución

El trabajo avanzó de lo más básico a lo más dependiente, probando cada pieza antes de construir la siguiente.

1. **Análisis y diseño inicial:** lectura del enunciado, identificación de entidades, supuestos y primer diagrama de clases.
2. **Cimientos:** lista genérica, fecha y medidor de recursos.
3. **Matriz de amistad** de bits, con conteo de amigos en común.
4. **Clases del dominio:** foto, álbum, mensaje, solicitud y usuario, con copia profunda y operadores sobrecargados.
5. **Texto propio,** al descubrir la restricción sobre la cadena estándar.
6. **Formato de archivos, generador de datos y carga,** para poder probar con más de 1000 usuarios.
7. **Plataforma con inicio de sesión y mejores amigos.**
8. **Funcionalidades** de amistades, sugerencias, perfil de un amigo y mensajes.

9. **Guardado de datos** al cerrar sesión.
10. **Pruebas de casos límite y robustez,** con sus correcciones.
11. **Diagrama de clases final,** actualizado con las clases y métodos que aparecieron durante el desarrollo.

Respecto del primer diagrama, el diseño final añade las clases `Texto` y `LectorLinea`, el operador de asignación en las clases con memoria dinámica, la fecha actual y la validación de fechas, un `Medidor` de miembros estáticos, la lista de usuarios que dieron like en cada foto y una `Plataforma` completa con sus métodos de carga, guardado y menú.

9. Limitaciones y trabajo futuro

- La memoria de la matriz de amistad crece con el cuadrado del número de usuarios; para escalas mucho mayores convendría una lista de adyacencia.
- Las contraseñas se almacenan en texto plano; en un sistema real deberían guardarse cifradas.
- El guardado reescribe los archivos de forma directa; una falla durante la escritura podría dejarlos incompletos. Una mejora sería escribir en archivos temporales y reemplazar los originales al terminar.
- Si el programa se cierra sin pasar por el cierre de sesión, se pierden los cambios de esa sesión.
- La clase `Plataforma` concentra demasiadas responsabilidades y podría dividirse.
- El límite de 512 caracteres cuenta bytes, por lo que con codificaciones de varios bytes por letra un mensaje con tildes podría contar más de lo esperado.
- Quedan fuera de alcance, según el enunciado, el registro de usuarios, la eliminación de cuentas y amistades, la edición de álbumes y fotos, y la respuesta directa a mensajes.

10. Uso de herramientas de inteligencia artificial

Con autorización del profesor, durante el desarrollo se empleó una herramienta de inteligencia artificial como apoyo para el análisis, la discusión del diseño, la depuración, la elaboración de pruebas y la redacción de este informe. Los integrantes revisaron, ejecutaron y comprendieron el código y las decisiones documentadas, y son responsables de su contenido.

11. Conclusiones

La solución cumple con las funcionalidades solicitadas utilizando únicamente estructuras de datos propias y memoria dinámica. La combinación de una matriz de bits para la amistad, búsquedas binarias sobre usuarios y mensajes, y un índice para los mensajes no leídos permitió mantener el costo de cada funcionalidad interactiva entre unas pocas iteraciones y algunos miles, con un

consumo de memoria cercano a 5 MB para 1200 usuarios. El aislamiento del tipo de texto, el desarrollo por capas y las pruebas sistemáticas, incluidas las de casos límite y archivos dañados, permitieron detectar y corregir defectos antes de la entrega. Como aprendizaje principal, el diseño cuidadoso de las estructuras antes de programar y la medición constante del consumo de recursos resultaron más determinantes para la calidad del resultado que la cantidad de código escrito.